

CSE 402

University of Asia Pacific

Course Name : Operating System Lab

Lab Report 3

Topic : Round Robin (Preemptive)

Presented by,

Fahad Hasan Mahi
Reg : 23101186
Section : D1

Presented to,

Atia Rahman Orthi
Lecturer, UAP

Problem Statement :

Consider the following processes, each represented as (Process ID, Arrival Time, Burst Time). The **time quantum is 5 units**. Using the preemptive **Round Robin** scheduling algorithm, calculate: Completion Time (CT), Turnaround Time (TAT = CT − AT), Waiting Time (WT = TAT − BT), and the average TAT and WT.

Process ID	Arrival Time (A.T)	Burst Time (B.T)
P1	0	7
P2	1	4
P3	2	15
P4	3	11
P5	4	20
P6	4	9

Solution (step-by-step simulation, TQ = 5):

Step	TIme	Process	Executed	Remaining BT	Queue after execution
1	0-5	P1	5	2	P2,P3,P4,P5,P6,P1
2	5-9	P2	4	0	P3,P4,P5,P6,P1
3	9-14	P3	5	10	P4,P5,P6,P1,P3
4	14-19	P4	5	6	P5,P6,P1,P3,P4
5	19-24	P5	5	15	P6,P1,P3,P4,P5
6	24-29	P6	5	4	P1,P3,P4,P5,P6
7	29-31	P1	2	0	P3,P4,P5,P6
8	31-36	P3	5	5	P4,P5,P6,P3
9	36-41	P4	5	1	P5,P6,P3,P4
10	41-46	P5	5	10	P6,P3,P4,P5
11	46-50	P6	4	0	P3,P4,P5
12	50-55	P3	5	0	P4,P5
13	55-56	P4	1	0	P5
14	56-61	P5	5	5	P5
15	61-66	P5	5	0	-

Source Code :

Round Robin (preemptive)-

```
def round_robin(parameters, quantum):
 n = len(parameters)
 pid = [p[0] for p in parameters]
 arrival = [p[1] for p in parameters]
 burst = [p[2] for p in parameters]
 remaining = burst.copy()

 completion = [0] * n
 turnaround = [0] * n
 waiting = [0] * n
 response = [0] * n
 first_run = [-1] * n

 order = sorted(range(n), key=lambda i: (arrival[i], pid[i]))
 queue = []
 gantt = []
 current_time = 0
 completed = 0
 ptr = 0

 while completed < n:
 # CPU idle: jump to next arrival
 if not queue:
 current_time = max(current_time, arrival[order[ptr]])
 queue.append(order[ptr])
 ptr += 1

 i = queue.pop(0)

 if first_run[i] == -1:
 first_run[i] = current_time
 response[i] = first_run[i] - arrival[i]

 # Run for at most one time quantum (preemption point)
 exec_time = min(quantum, remaining[i])
 start = current_time
 current_time += exec_time
 remaining[i] -= exec_time

 if gantt and gantt[-1][0] == pid[i] and gantt[-1][2] == start:
 gantt[-1] = (pid[i], gantt[-1][1], current_time)
 else:
 gantt.append((pid[i], start, current_time))
```

```
 # Newly arrived processes enter queue BEFORE current process re-joins
 while ptr < n and arrival[order[ptr]] <= current_time:
 queue.append(order[ptr])
 ptr += 1

 if remaining[i] == 0:
 completed += 1
 completion[i] = current_time
 turnaround[i] = completion[i] - arrival[i]
 waiting[i] = turnaround[i] - burst[i]
 else:
 queue.append(i)

 return {
 'pid': pid, 'arrival': arrival, 'burst': burst,
 'completion': completion, 'turnaround': turnaround,
 'waiting': waiting, 'response': response, 'gantt': gantt,
 'avg_tat': sum(turnaround) / n,
 'avg_wt': sum(waiting) / n,
 'avg_rt': sum(response) / n
 }


def display(r, name, quantum=None):
 title = name if quantum is None else f"{name} (Time Quantum = {quantum})"
 print(f"===== {title} =====")
 print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse")
 for i in range(len(r['pid'])):
 print(f"{r['pid'][i]}\t{r['arrival'][i]}\t{r['burst'][i]}\t"
 f"{r['completion'][i]}\t\t{r['turnaround'][i]}\t\t"
 f"{r['waiting'][i]}\t\t{r['response'][i]}")
 print("Gantt Chart:")
 for p, s, e in r['gantt']:
 print(f"[{p}: {s}->{e}]", end=" ")
 print(f"\nAverage Turnaround Time: {round(r['avg_tat'], 2)}")
 print(f"Average Waiting Time: {round(r['avg_wt'], 2)}")
 print(f"Average Response Time: {round(r['avg_rt'], 2)}")


parameters = [
 ('P1', 0, 7), ('P2', 1, 4), ('P3', 2, 15),
 ('P4', 3, 11), ('P5', 4, 20), ('P6', 4, 9)
]

 display(round_robin(parameters, 5), "Round Robin", 5)
```

(For SRTF/preemptive)-

```
def srtf(processes):
 n = len(processes)
 pid = [p[0] for p in processes]
 arrival = [p[1] for p in processes]
 burst = [p[2] for p in processes]
 remaining = burst.copy()

 completion = [0] * n
 turnaround = [0] * n
 waiting = [0] * n
 response = [0] * n
 start_time = [-1] * n
 is_completed = [False] * n

 current_time = 0
 completed = 0
 gantt_chart = []
 previous_pid = None

 while completed < n:
 shortest_idx = -1
 min_remaining = float('inf')

 for i in range(n):
 if (arrival[i] <= current_time
 and not is_completed[i]
 and remaining[i] < min_remaining):
 min_remaining = remaining[i]
 shortest_idx = i

 # CPU idle
 if shortest_idx == -1:
 if previous_pid != "IDLE":
 gantt_chart.append(("IDLE", current_time, current_time + 1))
 previous_pid = "IDLE"
 current_time += 1
 continue

 if start_time[shortest_idx] == -1:
 start_time[shortest_idx] = current_time

 current_time += 1 # run 1 unit, then re-check
 remaining[shortest_idx] -= 1

 if previous_pid != pid[shortest_idx]:
 gantt_chart.append((pid[shortest_idx], current_time - 1, current_time))
```

```
 else:
 gantt_chart[-1] = (pid[shortest_idx], gantt_chart[-1][1], current_time)
 previous_pid = pid[shortest_idx]

 if remaining[shortest_idx] == 0:
 is_completed[shortest_idx] = True
 completion[shortest_idx] = current_time
 turnaround[shortest_idx] = completion[shortest_idx] - arrival[shortest_idx]
 waiting[shortest_idx] = turnaround[shortest_idx] - burst[shortest_idx]
 completed += 1

 for i in range(n):
 response[i] = start_time[i] - arrival[i]

 return {'pid': pid, 'arrival': arrival, 'burst': burst,
 'completion': completion, 'turnaround': turnaround,
 'waiting': waiting, 'response': response,
 'gantt_chart': gantt_chart}


processes = [
 ('P1', 0, 7), ('P2', 1, 4), ('P3', 2, 15),
 ('P4', 3, 11), ('P5', 4, 20), ('P6', 4, 9)
]

result = srtf(processes)
n = len(processes)

print("===== SRTF (Preemptive SJF) =====")
print("Process\tArrival\tBurst\tCompletion\tTurnaround\tWaiting\tResponse")
for i in range(n):
 print(f"{result['pid'][i]}\t{result['arrival'][i]}\t{result['burst'][i]}\t"
 f"{result['completion'][i]}\t\t{result['turnaround'][i]}\t\t"
 f"{result['waiting'][i]}\t\t{result['response'][i]}")

print("Gantt Chart:")
for name, start, end in result['gantt_chart']:
 print(f"[{name}: {start}->{end}]", end=" ")
print()

print(f"\nAverage Turnaround Time: {round(sum(result['turnaround']) / n, 2)}")
print(f"Average Waiting Time: {round(sum(result['waiting']) / n, 2)}")
print(f"Average Response Time: {round(sum(result['response']) / n, 2)}")
```

Output Screenshot :

Round Robin (preemptive)

```
~/OS_Lab/CSE_402_Assignment_materials main*
❯ /usr/bin/python "/home/fahad369/OS_Lab/CSE_402_Assignment_materials/Lab 3/Round_Robin-(preemptive).py"
===== Round Robin (Time Quantum = 5) =====
Process Arrival Burst   Completion      Turnaround      Waiting Response
P1      0       7       31              31              24              0
P2      1       4       9               8               4               4
P3      2       15      55              53              38              7
P4      3       11      56              53              42              11
P5      4       20      66              62              42              15
P6      4       9       50              46              37              20
Gantt Chart:
[P1: 0->5] [P2: 5->9] [P3: 9->14] [P4: 14->19] [P5: 19->24] [P6: 24->29] [P1: 29->31] [P3: 31->36] [P4: 36->41] [P5: 41->
46] [P6: 46->50] [P3: 50->55] [P4: 55->56] [P5: 56->66]
Average Turnaround Time: 42.17
Average Waiting Time: 31.17
Average Response Time: 9.5
```

(for SRTF/preemptive)

```
===== SRTF (Preemptive SJF) =====
Process Arrival Burst   Completion  Turnaround  Waiting Response
P1  0   7   11      11      4       0
P2  1   4   5       4       0       0
P3  2   15  46      44      29      29
P4  3   11  31      28      17      17
P5  4   20  66      62      42      42
P6  4   9   20      16      7       7
Gantt Chart:
[P1: 0->1] [P2: 1->5] [P1: 5->11] [P6: 11->20] [P4: 20->31] [P3: 31->46]
[P5: 46->66]

Average Turnaround Time: 27.5
Average Waiting Time: 16.5
Average Response Time: 15.83

=== Code Execution Successful ===
```

Comparison between Round Robin (preemptive) & SJF (preemptive) :

Numeric :

Metric	Round Robin	SRTF
Average TAT	42.17	27.15
Average WT	31.17	16.5
Average RT	9.5	6.0

Feature Comparison :

Aspect	SJF (preemptive)	Round Robin
Selection	Shortest Remaining Burst Time	Order of arrival (cyclic)
Preemption Trigger	Shorter Job arrives	Time Quantum Expires
Starvation	Possible	Not Possible
Average Waiting Time	Minimum (optimal)	Higher than SRTF
Fairness	Not fair (prefers short jobs)	Fair
Use Case	Batch System, theoretical benchmark	Time Sharing/interactive system

Why SRTF (preemptive SJF) is better (efficiency perspective):

1. **Provably minimum waiting time:** SRTF always runs the job closest to completion, which greedily minimizes total waiting. On this input it cuts average WT nearly in half (16.5 vs 31.17).
2. **Fewer wasteful context switches:** RR switches on every quantum expiry whether it helps or not (13 switches here); SRTF switches only when beneficial (6 switches), so less CPU time is lost to overhead.
3. **Short jobs finish fast:** P2 completes at t=5 under SRTF but t=9 under RR; quick completion of short jobs empties the ready queue faster and improves throughput.
4. **No quantum-tuning problem:** RR's performance depends heavily on the quantum (too large → behaves like FCFS; too small → overhead dominates). SRTF has no such parameter to tune.
5. **Better turnaround overall:** Average TAT drops from 42.17 (RR) to 27.5 (SRTF) on identical input.

Discussion :

The two program outputs make the fairness - efficiency trade-off quantitative rather than theoretical. Under Round Robin (TQ = 5), every process received its first CPU slice by t = 24, and the worst first-response delay was 20 units (P6). Under SRTF, short jobs responded instantly (P2's response = 0), but the long job was locked out: P5 did not touch the CPU until t = 46, giving it a response time of 42 , nearly three times worse than its RR response of 15. This is why, on this dataset, RR's average response time (9.5) is actually better than SRTF's (15.83), even though SRTF wins decisively on waiting time (16.5 vs 31.17) and turnaround time (27.5 vs 42.17). The two algorithms are not "better/worse" versions of each other; they optimize different metrics.

A second observation concerns overhead. The RR Gantt chart contains 14 execution segments (13 context switches), and several of those switches are pure waste: P4 is preempted at t = 41 with only 1 unit of burst left, and P5's final two quanta (56–61 and 61–66) run back-to-back, so the switch at t = 61 accomplished nothing. SRTF completes the identical workload with only 7 segments (6 switches). RR therefore pays an overhead tax whenever the quantum expires at an unlucky moment, while SRTF switches only when it is useful.

Finally, the experiment exposes RR's parameter sensitivity. With TQ = 5 the schedule behaves reasonably, but a quantum of 20+ would collapse RR toward FCFS, while a quantum of 1–2 would explode the switch count. SRTF has no such tuning knob; its weaknesses are informational instead , it needs exact burst times in advance (rarely available outside batch systems), and its starvation risk, visible here in P5's 42-unit lockout, grows whenever short jobs keep arriving. *(On a personal note, the same trade-off appears in my Vortext extension's task queue: RR-style rotation keeps every UI task responsive, while SRTF-style prioritization minimizes total completion time of subtitle fetch/parse jobs.)*

Conclusion :

This lab demonstrated that "better" is metric-dependent when comparing preemptive schedulers. Round Robin produced weaker waiting and turnaround averages (31.17 / 42.17) but a stronger, more evenly distributed response time (9.5) and a hard guarantee that no process starves; SRTF produced near-optimal waiting and turnaround (16.5 / 27.5) at the price of locking out long processes and demanding burst-time knowledge that real systems rarely possess. Round Robin is therefore the safer default for interactive, time-sharing environments where every process must be assured progress, while SRTF remains the efficiency benchmark for offline workloads where burst lengths are known in advance. Rather than declaring a single winner, the results explain why practical operating systems adopt hybrid designs such as multilevel feedback queues, which rotate processes in RR fashion while dynamically promoting or demoting them based on observed burst behavior, capturing RR's fairness without fully giving up SRTF's efficiency.

Github repository : https://github.com/fhmahi957/CSE_402_Assignment_materials.git